

Forge

Habit Tracking & Budgeting with a Points Economy

Modern Application Development — Project Report

Course T22026_cs4010, App Dev Lab

Institution IIT Madras — BS Degree Programme

Author Anupratee

Roll Number 21f1001850

Submitted September 2026

1. Author

Anupratee — Roll Number 21f1001850, IIT Madras BS Degree Programme.

I built Forge end to end: schema, API, frontend, and the test suite. My approach was to settle each structural decision once and record it, so later phases could not quietly contradict earlier ones — the repository's `CLAUDE.md` is that running record of settled decisions. The two principles I held to throughout were that **the database, not the application, enforces anything that must never be violated**, and that **authorization is an API property** — the interface may hide a button, but the server is what refuses the request.

2. Description

Forge is a gamified self-improvement platform that puts personal habit tracking and monthly budgeting behind a single points economy, wrapped in a community layer of moderated challenges.

A user builds habits and checks in daily, sets a monthly spending limit per category and logs expenses against it, and joins time-boxed challenges. Consistency pays: completing a habit, holding a streak, staying inside a budget, or checking into a challenge mints points into an append-only ledger. Points are spent in a reward store on cosmetic themes the user can wear and simulated vouchers.

Three strictly separated roles share the platform. **Users** track their own private data and compete on an opt-in leaderboard. **Creators** author challenges and watch progress on the ones they own — and only those. **Admins** approve or reject challenges with a reason, curate the store, and govern accounts by suspension rather than deletion, so ledger history and challenge ownership stay readable.

3. Technologies Used

Layer	Technology	Why / how it is used
Backend	Node.js 24, Express 5	Express 5 forwards rejected promises to the error handler, so no handler needs a try/catch or an async wrapper.
Database	PostgreSQL 16 (Docker)	Unique keys, check constraints, and row locks carry the invariants; money is aggregated in SQL.
ORM	TypeORM 0.3 + SnakeNamingStrategy	Code-first: entity classes are the only source of the schema; migrations are generated, never hand-written.
Auth	JWT (jsonwebtoken), bcrypt cost 12	The token carries a role for the client's benefit only; the API re-reads role and status every request.
Validation	class-validator / class-transformer	One validate(Dto) middleware per route with whitelist + forbidNonWhitelisted .
Frontend	React 19, React Router 7, TanStack Query 5, Axios	Server state lives in React Query; one hook per read and per write, one query-key registry.
Security	helmet, cors, express-rate-limit	Auth endpoints are rate-limited; the limiter skips only under NODE_ENV=test and is proved by its own test.
Uploads / AI	multer, @anthropic-ai/sdk , zod, csv-parse	Image uploads to disk; bank statements read in memory and parsed into schema-validated rows.
Testing / CI	Vitest (SWC transform), supertest, GitHub Actions	280 tests, integration ones against a separate forge_test database built by the real migrations.
Tooling	TypeScript (strict), ESLint, Prettier, Docker Compose, Vite	Lint, typecheck, test, and build gate every push.

Redis is wired in as an optional cache profile, and the AI import self-disables without an API key — both are reported by `GET /health` .

4. Database Schema Design

Eleven tables, all generated from TypeORM entity classes. Two abstract bases divide them by nature: **AuditedEntity** (id, created_at, updated_at) for mutable records, and **AppendOnlyEntity** (id, created_at) for events — a ledger row or a check-in has nothing to update, so it is not given the column.

Table	Key columns	Constraints that matter
users	email, password_hash, display_name, role, status, leaderboard_opt_in, equipped_redemption_id	Unique email; password_hash is select: false; equipping points at a <i>Redemption</i> , not a reward — you can only wear what you bought.
challenges	title, category, start_date, end_date, capacity, points_reward, status, created_by, approved_by	end_date > start_date, capacity > 0, points_reward ≥ 0; index on (status, category).
challenge_participations	challenge_id, user_id, joined_at	Unique (challenge, user); capacity is enforced under a pessimistic row lock, not a count-then-insert.
challenge_check_ins	participation_id, check_in_date	Unique (participation, date) — one payout per day, guaranteed by the database.
habits	user_id, name, category, target_per_week, is_archived	target_per_week BETWEEN 1 AND 7.
habit_completions	habit_id, completed_on	Unique (habit, date) . Streaks are computed, never stored.
budget_goals	user_id, category, period_month, limit_amount	Unique (user, month, category); period_month must be a month start; limit > 0.
expenses	user_id, title, amount, category, spent_on, source	amount > 0; source is MANUAL / CSV_IMPORT / AI_IMPORT and is always set by the server.
reward_items	name, type, points_cost, stock, is_active, cosmetic_theme (jsonb)	cost > 0, stock ≥ 0.
redemptions	user_id, reward_item_id, points_spent, voucher_code	Append-only; points_spent > 0.
points_ledger	user_id, amount (signed), reason, reference_type, reference_id	Unique (reference_type, reference_id, reason) and amount <> 0. The only source of truth for a balance.

Design decisions

- **Calendar dates are YYYY-MM-DD strings, never Date**. A SQL date has no zone; mapping it to a JavaScript Date shifts the day off UTC, which would be a correctness bug in check-ins, streaks, and budget months — not a formatting one. A custom pg type parser keeps raw queries consistent with entity reads.
- **Money is numeric(12,2) and is never summed in JavaScript**. PostgreSQL does the arithmetic.
- **Every relation also declares its FK id column**, so an ownership check is row.userId !== actorId with no join.
- **No foreign key from expenses to budget_goals** — they meet on the natural (user, month, category) key, so a budget can be set before or after the spending it measures.
- **Point values live in a service policy**, not on entities; only a Creator-set, Admin-approved points_reward is stored per challenge.

5. API Design

Every endpoint sits under `/api`. One error middleware is the only place a status code is set: services throw typed `AppError` subclasses (`NotFoundError`, `ForbiddenError`, `ConflictError`) and each maps to one status and one envelope, `{ code, message, details? }`. Every list endpoint reuses one query DTO — `keyword`, `category`, `dateFrom`, `dateTo`, `sortBy`, `sortDir`, `page`, `pageSize`, `availableOnly` — and returns one page envelope; `sortBy` is an allow-list mapped to a column, so a query value never reaches an `ORDER BY`.

Group	Representative routes	Access
Authentication	<code>POST /auth/register</code> , <code>POST /auth/login</code> , <code>GET PATCH /auth/me</code>	Public (rate-limited) / signed in
Challenges — browse	<code>GET /challenges</code> , <code>GET /challenges/:id</code>	Any; unapproved visible only to author and Admin
Challenges — author	<code>POST /challenges</code> , <code>PATCH DELETE /challenges/:id</code> , <code>/submit</code> , <code>/:id/participants</code>	Creator, own challenges only
Challenges — moderate	<code>GET /challenges/pending-approval</code> , <code>POST /:id/approve reject</code>	Admin
Challenges — participate	<code>GET /challenges/joined</code> , <code>POST /:id/join</code> , <code>POST /:id/check-ins</code>	User
Habits	<code>/habits</code> , <code>/habits/:id</code> , <code>/habits/:id/completions</code>	User — private, owner only
Budgets & expenses	<code>/budgets</code> , <code>/budgets/summary</code> , <code>/budgets/:id/adherence-claim</code> , <code>/expenses</code>	User — private, owner only
Expense import	<code>/expenses/import/csv</code> , <code>/import/statement</code> , <code>/import/confirm</code>	User
Store	<code>GET /rewards</code> ; <code>/rewards/manage</code> ; <code>POST /rewards/:id/redeem</code> , <code>PUT /rewards/equipped</code>	Any / Admin / User
Points & standing	<code>/points/balance</code> , <code>/points/ledger</code> , <code>/leaderboard</code>	Any — always the caller's own
Administration	<code>/admin/summary</code> , <code>/admin/users</code> , <code>PATCH /admin/users/:id/status role</code>	Admin

A 404, not a 403, for another user's private data. A 403 confirms the row exists, and existence is itself the leak when the resource is somebody's habit or budget. Where existence is already public — an approved challenge's participant list — a 403 is correct, because there is nothing left to hide.

6. Architecture and Features

Architecture

`server/` and `client/` are fully independent packages with separate installs, communicating only over HTTP. Inside the server, dependencies point one way and nothing skips a layer:

```
route → middleware → controller → service → repository
      (authenticate, (HTTP in/out (every business rule,
      authorize,      only)         every transaction)
      validate)
```

- **Controllers** translate an HTTP request into a service call and back — no query building, no role checks.
- **Services** own every business rule and every transaction, and never touch `req/res`, which is what keeps them unit-testable.
- **Authorization** is coarse role gates at the route (`authorize(Role.ADMIN)` , listed explicitly per route) plus ownership checks inside services. No rule is checked in two layers.
- **The client** mentions a URL in `services/` only; pages compose components and call hooks and hold no business logic. `RequireRole` is UX and says so in its own docblock.

Features

- **Habits & streaks** — daily completions with a weekly target; streaks are derived by a pure, unit-tested calculator, never stored.
- **Budgets & expenses** — per-category monthly limits, expenses logged with receipts, adherence claimed at month end and paid out in points.
- **Challenges** — `DRAFT → PENDING_APPROVAL → APPROVED → ENDED` , with `REJECTED` returning to the queue after an edit. Every transition goes through one state machine file, so "a Creator cannot self-approve" is stated once rather than re-implemented per endpoint, and a material edit to an approved challenge re-enters approval.
- **Points economy** — an append-only ledger is the only source of a balance; nothing caches one, so there is no second copy to drift.
- **Store, cosmetics & leaderboard** — redeem for themes and vouchers, wear one cosmetic at a time (structural, via a single column), and rank with SQL `RANK()` over opted-in accounts only — opting out removes the row from the query, not from the response.
- **Statement import (optional AI)** — CSV or a PDF bank statement sent to the model as a document, both feeding one pipeline. Preview writes nothing; confirm writes and *re-validates*, because the preview is a suggestion and the rows come back edited by design. Without an API key the endpoint answers 503 and the UI offers CSV.
- **Administration** — accounts are suspended, never deleted; the only refusals are self-suspend and self-role-change, which is what guarantees an active Admin always remains.

Invariants enforced by PostgreSQL, not by application checks

An application check is a read followed by a write, and two concurrent requests both pass the read. So:

- **An action pays out once** — unique keys on `(habit_id, completed_on)` , `(participation_id, check_in_date)` and `(reference_type, reference_id, reason)` . The triggering row and the ledger row are written in one transaction; a duplicate is caught as a unique violation and rethrown as a 409.
- **Spending cannot exceed the balance** — redemption locks the user's row, sums the ledger, then inserts. Without the lock two simultaneous redemptions both see enough and both succeed.
- **Capacity cannot be exceeded** — the same pessimistic lock, not a count-then-insert.

7. Testing and Quality

280 tests run under Vitest. Integration tests hit the real API through supertest against a separate `forge_test` database, created on first run and built by applying the same generated migrations the development database uses — never by `synchronize`. The harness refuses any database whose name does not end in `_test`, because it truncates every domain table between files, and tests build their own fixtures rather than reading seeded data.

Suite	What it proves
<code>rbac.test.ts</code>	224 checks: every protected route refuses every role it does not name — and admits the ones it does , since a route answering 403 to everyone would otherwise pass. Also: suspension and demotion take effect on the next request, not at token expiry; role and status cannot be smuggled through a profile update.
<code>points.test.ts</code>	The economy invariants, fired <i>concurrently</i> — duplicate check-ins and over-balance redemptions as simultaneous pairs, because a sequential test passes against an implementation with neither guarantee.
<code>auth.test.ts</code>	bcrypt storage, an identical answer for a wrong password and an unknown email, refused sign-in while suspended.
<code>malformed-requests.test.ts</code>	A client mistake is never a 5xx — bad bodies, bad ids, and body-parser failures all come back as 4xx that say what was wrong.
<code>StreakCalculator</code> , <code>date</code> , <code>rate-limit</code>	Pure logic, including the calendar boundaries naive day arithmetic gets wrong, and the auth limiter refusing past its limit.

The RBAC matrix is transcribed by hand rather than derived from the router — a matrix generated from the same `authorize` calls it checks would still pass with a guard deleted. GitHub Actions runs lint, typecheck, tests, and build for the server and lint, typecheck, and build for the client on every push and pull request.

8. Running the Project

```
cp .env.example .env      # set JWT_SECRET – the placeholder is refused at boot
docker compose up -d      # postgres:16 on 5432

cd server && npm install && npm run migration:run && npm run seed && npm run dev    # API   :3000
cd client && npm install && npm run dev                                           # SPA   :5173
```

Open the SPA at `http://localhost:5173`. A missing or malformed `.env` fails at boot with every problem listed at once, rather than one `undefined` per restart. Seeded accounts — `admin@`, `maya@/dev@` (Creators), `asha@/rohan@`, and `kim@` (suspended) at `@forge.test` — all use the password `Forge!2026`.

AI assistance was used throughout development; `ai_usage.md` in the repository records what was asked for at each phase, what was accepted, and what was rejected or corrected on review.